Global Student Portal

Design for a worldwide student portal platform, inspired by `wsp.kbtu.kz`, but built as a **multi-tenant SaaS** where many universities can onboard independently while sharing one core platform.

build one secure, scalable platform where students can:

- view schedules, grades, GPA, transcript, attendance
- register for courses and add/drop subjects
- pay tuition and fees
- access LMS, library, dormitory, and support services
- receive notifications and submit requests

and universities can:

- onboard with their own branding, policies, grading rules, and academic calendar
- integrate with local SIS/ERP/LMS/payment systems
- manage users, roles, and analytics per institution

---

1. requirements

functional

- multi-institution onboarding (tenant per university)
- Student/Faculty/Advisor/Admin portals
- course catalog + registration + waitlist
- timetable + attendance + exam schedule
- grades + GPA + transcript generation
- tuition invoicing + payment tracking
- ticketing/requests (e.g., transcript request, enrollment verification)
- notifications: in-app, email, push, SMS
- university-level CMS (announcements, deadlines, news)

non-functional

- 99.95% platform availability
- P95 API latency < 300ms for read-heavy endpoints
- strong data isolation between universities
- regional data residency options (EU, US, APAC, ME)
- auditability for grade/payment operations
- horizontal scaling for peak enrollment periods

---

2. mlti-tenant strategy

tenant model

- **tenant = university**
- unique tenant ID in every request and data row
- optional custom domain per tenant:
  - `portal.university-a.edu`
  - `portal.university-b.edu`

data isolation approach

Use **hybrid isolation**:

1. shared database cluster with `tenant_id` row-level isolation for most tenants
2. dedicated database/schema option for premium or regulated tenants

why hybrid:

- shared mode lowers cost and simplifies operations
- dedicated mode satisfies strict compliance/data-residency contracts

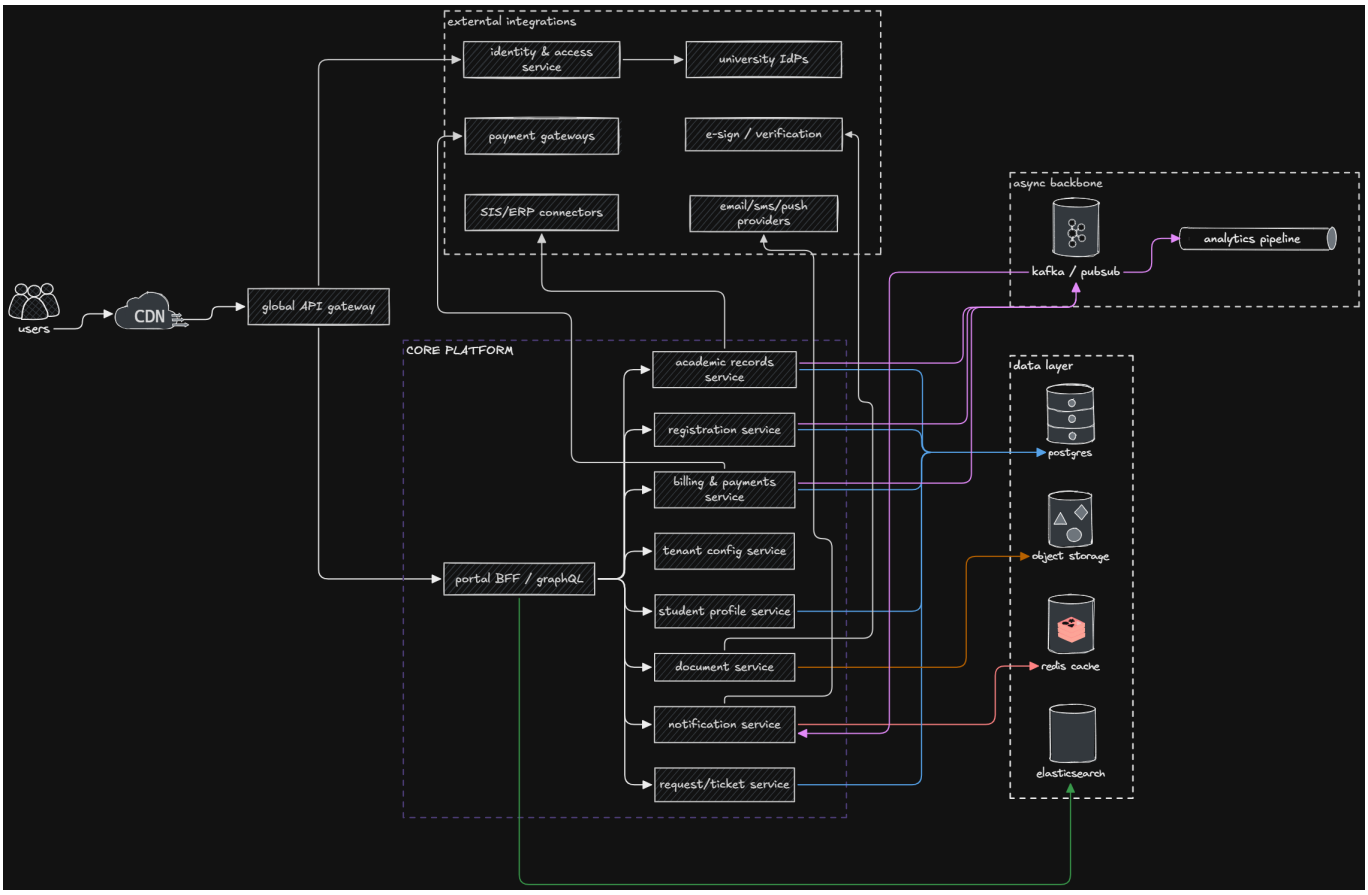
tenant configuration

per-tenant config service stores:

- grading scale and GPA formula
- add/drop policy windows
- semester/trimester structure
- timezone + locale + language
- SSO provider (Azure AD, Google, SAML, OIDC)
- payment providers and currency

---

3. high-level architecture



---

4. service breakdown

4.1 identity & access service

- supports tenant-aware authentication and SSO federation
- issues JWT with claims: `tenant_id`, `user_id`, `roles`, `permissions`
- enforces MFA for admin and sensitive actions
- RBAC + ABAC (role + policy conditions, e.g., advisor can only view assigned students)

4.2 student profile service

- student demographic profile, contacts, emergency info
- program/major/minor and advisor mapping
- privacy flags and consent preferences

4.3 academic records service

- course enrollments, grades, GPA, transcripts
- grade change workflow with approvals and immutable audit logs
- transcript rendering + signed PDF generation

4.4 registration service

- catalog, prerequisites, section capacity, timetable conflict checks
- registration transaction model with seat lock (short TTL)
- waitlist auto-promotion via event-driven rules

4.5 billing & payments service

- Tuition and fee ledger
- Invoice generation by academic term
- Gateway abstraction (Stripe/Adyen/local providers)
- Idempotent payment callbacks and reconciliation jobs

4.6 request/ticket service

- handles student service requests: certificates, appeals, deferrals
- SLA and queue assignment by department
- workflow states + notifications

4.7 notification service

- template-based messaging by channel and language
- event-driven notifications (registration success, deadline reminders)
- delivery tracking + retry policy + dead-letter queue

4.8 document service

- store generated docs (transcripts, invoices, enrollment letters)
- signed URL access with short expiry
- optional digital signature integration

4.9 tenant config service

- tenant bootstrap and runtime feature flags
- policy and calendar management
- branding assets and UI customization metadata

---

5. critical workflows

5.1 course registration (high contention)

1. student chooses section
2. registration service validates prerequisites/conflicts
3. seat is locked in Redis (`seat_lock:{section}:{student}` TTL 30s)
4. enrollment transaction commits in DB
5. event published: `enrollment.created`
6. notification + analytics consumers process asynchronously

failure handling:

- if payment-required registration fails after seat lock, lock expires automatically
- compensating event `enrollment.cancelled` updates downstream projections

5.2 grade submission

1. faculty submits grades

2. rules engine validates grading policy for tenant
3. department approval (if configured)
4. grade freeze event creates immutable audit snapshot
5. GPA recalculation triggered asynchronously

5.3 tuition payment

1. invoice selected; payment intent created
2. redirect/embedded checkout via tenant gateway adapter
3. callback webhook verifies signature and idempotency
4. payment applied to ledger; receipt generated
5. student notified with receipt and updated balance

---

6. security, privacy, compliance

- OIDC/SAML SSO and short-lived JWT tokens
- at-rest encryption (KMS) + TLS 1.2+
- fine-grained authorization policies (OPA or equivalent)
- audit logs for grade/payment/profile changes
- PII data classification and field-level encryption for sensitive fields
- data retention policies by tenant and region
- compliance readiness: GDPR/FERPA-style controls (consent, export, deletion requests)

---

7. scalability & performance

- read-heavy endpoints served with Redis cache and read replicas
- CQRS-style read models for dashboard aggregation
- async event bus decouples heavy post-processing
- autoscaling by metrics: CPU, queue lag, request rate
- enrollment-week protections:
    - rate limits per tenant
    - priority queues for core registration APIs
    - adaptive backpressure and graceful degradation

---

8. availability, DR, operations

deployment topology

- multi-region active-active for frontend/API gateway
- core data in primary region per tenant; cross-region replicas for DR
- RPO <= 5 minutes, RTO <= 30 minutes (target)

observability

- centralized logs + distributed tracing + metrics
- SLO dashboards by tenant and by service

- alerting for auth failures, payment callback lag, enrollment error spikes

operational model

- blue/green or canary deployments
- schema migration with backward-compatible versioning
- feature flags per tenant for safe rollout